

CS 4375 - Intro to Machine Learning Project

Neural Network Library from Scratch CS 4375 -- Prof. Rishabh Iyer | UTD

1. Introduction

Modern deep learning is dominated by frameworks like PyTorch and TensorFlow that abstract away the mechanics of automatic differentiation, GPU memory management, and optimizer state. While these tools are necessary in practice, they make it easy to treat neural networks as black boxes. As a result, we don't understand what happens during a forward pass, why a gradient vanishes, or how Adam's bias correction actually works.

This project takes the opposite approach: building a fully functional neural network library from scratch, using only NumPy (CPU) and CuPy (GPU) as compute backends using no autograd and no framework primitives. Every gradient is derived analytically and implemented by hand.

1.1. What Was Built

The library implements the following components entirely from scratch:

- **Layers:** Fully connected (Dense) with He/Xavier initialization, Dropout (inverted), Batch Normalization
- **Activations:** ReLU, Leaky ReLU, Sigmoid, Tanh, Softmax
- **Loss Functions:** Mean Squared Error (MSE), Binary Cross-Entropy (BCE), Categorical Cross-Entropy (CCE)
- **Optimizers:** Vanilla SGD, SGD with Momentum, Adam (with bias correction)
- **Training loop:** Mini-batch gradient descent with shuffling, validation loss tracking, and training/inference mode switching

The library exposes a clean sequential API modeled after Keras:

```
model = NeuralNetwork(loss='cce', optimizer=Adam(lr=0.001))
model.add(DenseLayer(784, 512, activation='relu'))
model.add(BatchNormLayer(512))
model.add(DropoutLayer(keep_prob=0.8))
model.add(DenseLayer(512, 10, activation='softmax'))
model.fit(X_train, y_train, epochs=20, batch_size=256)
```

1.2. Hardware & Backend

A key design decision was to build the library backend-agnostic. By substituting CuPy for NumPy (a drop-in replacement that dispatches matrix operations to CUDA), the entire library

runs on a GPU without code changes. A hardware detection routine automatically selects the appropriate backend at runtime:

```
try:
    import cupy as xp
    GPU_AVAILABLE = True
except Exception:
    import numpy as xp
    GPU_AVAILABLE = False
```

All experiments were run locally on an **NVIDIA RTX 5070 (Blackwell architecture, 8GB VRAM)** with CUDA 12.6 and CuPy `cupy-cuda12x`. A raw matrix operation benchmark demonstrated a **14.1× speedup** over CPU (NumPy), confirming that CuPy fully saturates the RTX 5070's tensor cores for the matrix multiplications that dominate neural network training.

1.3. Validation Strategy

The library is validated against scikit-learn's `MLPClassifier` and `MLPRegressor` across three datasets spanning classification and regression tasks. Results confirm that the from-scratch implementation achieves accuracy and loss within 1% of sklearn on binary classification, **outperforms sklearn on regression** (RMSE 0.4437 vs 0.4540, where sklearn failed to converge), and reaches 98.19% accuracy on the full 70,000-sample MNIST dataset.

2. Library Architecture

2.1. Design Philosophy

The library was built around three core principles:

- **Modularity.** Every component (layer, activation, loss, optimizer) is an independent class with a consistent interface. Adding a new activation or optimizer requires no changes to existing code.
- **Backend agnosticism.** All array operations go through a single namespace `xp`, which resolves to either CuPy or NumPy at runtime. This means the entire library runs on GPU or CPU without a single conditional branch in the core logic.
- **sklearn compatibility.** Data flows as NumPy arrays at the boundary (inputs and outputs), with conversion to the active backend handled internally. This makes it trivial to use sklearn for preprocessing, splitting, and evaluation while the library handles training.

2.2 Component Overview

The library consists of five layers of abstraction:

```

NeuralNetwork (model)
├─ DenseLayer          -- learnable weights, forward + backward
├─ BatchNormLayer      -- normalization, learnable gamma/beta
├─ DropoutLayer        -- inverted dropout mask
├─ Activation           -- ReLU, LeakyReLU, Sigmoid, Tanh, Softmax
├─ Loss                -- MSE, BCE, CCE
└─ Optimizer           -- SGD, SGDMomentum, Adam

```

Each layer exposes two methods:

- `forward(X, training)` - computes output and caches values needed for backprop
- `backward(dA)` - receives upstream gradient, computes parameter gradients and returns downstream gradient

Optimizers expose a single method:

- `update(layer, t)` - applies parameter update using the layer's cached `dW` and `db`; `t` is the global step count used by Adam for bias correction

2.3 Dual Backend Design

The backend selection happens once at import time:

```

try:
    import cupy as xp
    xp.array([1])          # confirm CUDA is live
    GPU_AVAILABLE = True
except Exception:
    import numpy as xp
    GPU_AVAILABLE = False

```

Two utility functions handle boundary conversions:

```

def to_xp(arr):
    # numpy -> cupy (if GPU available), else passthrough
    return xp.array(arr) if GPU_AVAILABLE else arr

def to_np(arr):
    # cupy -> numpy (for sklearn / matplotlib), else passthrough
    return xp.asnumpy(arr) if GPU_AVAILABLE else np.array(arr)

```

All internal operations (matrix multiply, elementwise ops, reductions) use `xp` directly, so they execute on whichever device is active. The RTX 5070 with CUDA 12.6 and `cupy-cuda12x` was

used for all experiments reported in this project.

2.4 The NeuralNetwork Class

The top-level `NeuralNetwork` class manages the layer stack, training loop, and optimizer dispatch. Key design decisions:

- **Layer chaining.** `model.add()` returns `self`, enabling fluent chaining:

```
model.add(DenseLayer(784, 256, activation='relu')).add(DropoutLayer(0.8))
```

- **Training vs inference mode.** Every layer's `forward()` receives a `training` flag. Dropout zeroes activations only during training; BatchNorm uses batch statistics during training and running statistics during inference. The flag propagates automatically through `_forward()`.
- **Global step counter.** A single integer `t` increments once per batch and is passed to the optimizer. Adam requires this for bias correction; SGD and SGDMomentum ignore it. This avoids per-layer step tracking.
- **Optimizer dispatch.** The update loop skips layers with no learnable parameters (Dropout) by checking `layer.W is not None`:

```
for layer in self.layers:
    if hasattr(layer, 'W') and layer.W is not None and layer.dW is not None:
        self.optimizer.update(layer, t)
```

`BatchNormLayer` exposes its learnable parameters `gamma` and `beta` as `W` and `b` respectively, so the same optimizer loop updates them without any special casing.

3. Mathematical Foundations

3.1 Forward Pass

For a network with L layers, the forward pass computes activations layer by layer. For layer l :

$$z^{(l)} = a^{(l-1)}W^{(l)} + b^{(l)}$$

$$a^{(l)} = f^{(l)}(z^{(l)})$$

where $W^{(l)} \in \mathbb{R}^{n_{l-1} \times n_l}$ is the weight matrix, $b^{(l)} \in \mathbb{R}^{1 \times n_l}$ is the bias, $f^{(l)}$ is the activation function, and $a^{(0)} = X$ is the input batch of shape (N, n_0) .

The final layer output $\hat{y} = a^{(L)}$ is passed to the loss function.

3.2 Activation Functions

ReLU

$$f(z) = \max(0, z)$$

$$f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

ReLU is used for all hidden layers by default. It avoids the vanishing gradient problem present in sigmoid/tanh, but neurons with $z \leq 0$ produce zero gradient permanently (dying ReLU).

Leaky ReLU

$$f(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0, \quad \alpha = 0.01 \end{cases}$$

$$f'(z) = \begin{cases} 1 & z > 0 \\ \alpha & z \leq 0 \end{cases}$$

Leaky ReLU fixes the dying ReLU problem by allowing a small gradient for negative inputs.

Sigmoid

$$f(z) = \frac{1}{1 + e^{-z}}$$

$$f'(z) = f(z)(1 - f(z))$$

Used at the output layer for binary classification. Numerically stabilized by clipping z to $[-500, 500]$ before exponentiation.

Tanh

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$f'(z) = 1 - \tanh^2(z)$$

Zero-centered output in $(-1, 1)$, which can improve convergence compared to sigmoid for hidden layers.

Softmax

$$f(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Used at the output layer for multiclass classification. Numerically stabilized by subtracting the row maximum before exponentiation:

$$f(z)_i = \frac{e^{z_i - \max_j z_j}}{\sum_j e^{z_j - \max_j z_j}}$$

The backward pass for Softmax is handled jointly with the Categorical Cross-Entropy loss (see Section 3.4), which simplifies the gradient to $\hat{y} - y$.

3.3 Backpropagation

Backpropagation applies the chain rule to compute gradients of the loss $\mathcal{L}$ with respect to every parameter in the network. Define the error signal at layer l as:

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}}$$

Output layer. Given the upstream gradient $\frac{\partial \mathcal{L}}{\partial a^{(L)}}$ from the loss function:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \odot f'^{(L)}(z^{(L)})$$

Hidden layers. Propagating backwards from layer $l + 1$ to layer l :

$$\delta^{(l)} = \left(\delta^{(l+1)} (W^{(l+1)})^T \right) \odot f'^{(l)}(z^{(l)})$$

Parameter gradients. For a batch of N samples:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{1}{N} (a^{(l-1)})^T \delta^{(l)} + \lambda W^{(l)}$$

$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \frac{1}{N} \sum_{i=1}^N \delta_i^{(l)}$$

The $\lambda W^{(l)}$ term is the L2 regularization gradient. When $\lambda = 0$ it vanishes and we recover standard backprop.

In the implementation, each `DenseLayer.backward(dA)` receives $\frac{\partial \mathcal{L}}{\partial a^{(l)}}$ as `dA`, computes $\delta^{(l)}$ by applying the activation derivative, computes `dW` and `db`, and returns $\frac{\partial \mathcal{L}}{\partial a^{(l-1)}}$ to the previous layer.

3.4 Loss Functions

Mean Squared Error (MSE)

Used for regression tasks.

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{2}{N} (\hat{y} - y)$$

Binary Cross-Entropy (BCE)

Used for binary classification with a sigmoid output.

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{1}{N} \left(-\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} \right)$$

Clipping $\hat{y}$ to $[10^{-9}, 1 - 10^{-9}]$ prevents $\log(0)$.

Categorical Cross-Entropy (CCE) with Softmax

Used for multiclass classification. The Softmax Jacobian is expensive to compute explicitly, so we use the combined Softmax + CCE gradient:

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{y}_{ik}$$

The combined gradient with respect to the pre-softmax logits $z^{(L)}$ simplifies to:

$$\frac{\partial \mathcal{L}}{\partial z^{(L)}} = \frac{1}{N} (\hat{y} - y)$$

This is the gradient returned by `CategoricalCrossEntropy.backward()` and passed directly into the output layer's backward pass, bypassing the Softmax Jacobian entirely.

3.5 Optimizers

Vanilla SGD

$$W \leftarrow W - \eta \cdot \frac{\partial \mathcal{L}}{\partial W}$$

where η is the learning rate. It is simple but sensitive to learning rate choice and slow to converge on ill-conditioned loss surfaces.

SGD with Momentum

Momentum accumulates a velocity vector v that dampens oscillations and accelerates convergence in consistent gradient directions:

$$v_W \leftarrow \beta v_W + \eta \cdot \frac{\partial \mathcal{L}}{\partial W}$$
$$W \leftarrow W - v_W$$

where $\beta = 0.9$ is the momentum coefficient. The learning rate η is absorbed directly into the velocity, so the effective step size grows over consistent gradient directions up to $\frac{\eta}{1-\beta}$.

Adam (Adaptive Moment Estimation)

Adam maintains per-parameter first and second moment estimates:

$$m_W \leftarrow \beta_1 m_W + (1 - \beta_1) \frac{\partial \mathcal{L}}{\partial W} \quad (\text{first moment})$$

$$v_W \leftarrow \beta_2 v_W + (1 - \beta_2) \left(\frac{\partial \mathcal{L}}{\partial W} \right)^2 \quad (\text{second moment})$$

Since m_W and v_W are initialized to zero, they are biased toward zero in early iterations. Bias correction compensates:

$$\hat{m}_W = \frac{m_W}{1 - \beta_1^t}, \quad \hat{v}_W = \frac{v_W}{1 - \beta_2^t}$$

The parameter update is:

$$W \leftarrow W - \frac{\eta}{\sqrt{\hat{v}_W} + \epsilon} \cdot \hat{m}_W$$

Default hyperparameters: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

The division by $\sqrt{\hat{v}_W}$ gives Adam an adaptive per-parameter learning rate: parameters with consistently large gradients get smaller updates, and parameters with small or infrequent gradients get relatively larger updates. This makes Adam particularly effective on sparse or high-dimensional problems like MNIST.

3.6 Batch Normalization

Batch Normalization normalizes layer inputs per mini-batch to stabilize training and allow higher learning rates. For a batch of activations $X \in \mathbb{R}^{N \times d}$:

Forward pass (training):

$$\mu_B = \frac{1}{N} \sum_{i=1}^N x_i$$

$$\sigma_B^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \odot \hat{x}_i + \beta$$

where γ and β are learnable scale and shift parameters, and $\epsilon = 10^{-8}$ ensures numerical stability. Running mean and variance are tracked via exponential moving average for use at inference time:

$$\mu_{\text{run}} \leftarrow \rho \cdot \mu_{\text{run}} + (1 - \rho) \cdot \mu_B$$

$$\sigma_{\text{run}}^2 \leftarrow \rho \cdot \sigma_{\text{run}}^2 + (1 - \rho) \cdot \sigma_B^2$$

with momentum $\rho = 0.9$.

Forward pass (inference): Uses running statistics instead of batch statistics so predictions are deterministic and independent of batch size.

Backward pass: The gradient through the normalization is non-trivial because μ_B and σ_B^2 both depend on x_i . The full derivation gives:

$$\frac{\partial \mathcal{L}}{\partial \gamma} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial y_i} \odot \hat{x}_i$$

$$\frac{\partial \mathcal{L}}{\partial \beta} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial y_i}$$

$$\frac{\partial \mathcal{L}}{\partial x_i} = \frac{1}{N} \cdot \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} \left(N \cdot \frac{\partial \mathcal{L}}{\partial \hat{x}_i} - \sum_j \frac{\partial \mathcal{L}}{\partial \hat{x}_j} - \hat{x}_i \sum_j \frac{\partial \mathcal{L}}{\partial \hat{x}_j} \hat{x}_j \right)$$

where $\frac{\partial \mathcal{L}}{\partial \hat{x}_i} = \frac{\partial \mathcal{L}}{\partial y_i} \odot \gamma$.

3.7 Dropout

Dropout randomly zeroes a fraction of activations during training to prevent co-adaptation of neurons. The inverted dropout variant used here scales the surviving activations by $\frac{1}{p}$ during training, so the expected activation magnitude is unchanged and no scaling is needed at inference:

Training:

$$m_i \sim \text{Bernoulli}(p)$$

$$\tilde{a}_i = \frac{a_i \cdot m_i}{p}$$

Inference:

$$\tilde{a}_i = a_i$$

Backward pass: The gradient is masked and rescaled identically:

$$\frac{\partial \mathcal{L}}{\partial a_i} = \frac{\partial \mathcal{L}}{\partial \tilde{a}_i} \cdot \frac{m_i}{p}$$

The mask m_i is cached during the forward pass and reused in the backward pass.

3.8 Weight Initialization

Poor weight initialization causes vanishing or exploding gradients before training even begins. The library selects initialization based on the downstream activation:

He initialization (for ReLU and Leaky ReLU):

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

The factor of 2 accounts for ReLU zeroing half the neurons in expectation, keeping the variance of activations stable across layers.

Xavier initialization (for Sigmoid, Tanh, Softmax):

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{1}{n_{\text{in}}}}\right)$$

Designed for symmetric activations where the full input variance is preserved. Biases are initialized to zero in all cases.

4. Implementation Details

This section covers the practical engineering decisions made during implementation: numerical stability tricks, training/inference mode handling, and design choices that are not obvious from the math alone.

4.1 Numerical Stability

Several components require explicit stabilization to avoid overflow, underflow, or division by zero.

Softmax: Exponentiating large logits causes overflow. The standard fix subtracts the row maximum before computing the exponential:

```
shifted = z - xp.max(z, axis=1, keepdims=True)
exp_z = xp.exp(shifted)
return exp_z / xp.sum(exp_z, axis=1, keepdims=True)
```

This is mathematically equivalent to the original formula since the constant cancels in the numerator and denominator, but keeps all values in $(-\infty, 0]$ before exponentiation.

Sigmoid: Very large negative inputs cause e^{-z} to overflow. Clipping z to $[-500, 500]$ before the exponential prevents this without affecting any realistically encountered activations:

```
return 1.0 / (1.0 + xp.exp(-xp.clip(z, -500, 500)))
```

BCE and CCE: Logarithms of zero or negative values are undefined. Both loss functions clip $\hat{y}$ to a small positive range before computing log:

```
y_hat = xp.clip(y_hat, 1e-9, 1 - 1e-9) # BCE
y_hat = xp.clip(y_hat, 1e-9, 1.0)      # CCE
```

Batch Normalization: Division by the standard deviation requires adding $\epsilon = 10^{-8}$ to the variance to prevent division by zero when all values in a feature are identical:

```
x_hat = x_mu / xp.sqrt(var + self.eps)
```

4.2 Training vs Inference Mode

Two components behave differently during training and inference: Dropout and Batch Normalization. The `training` flag is passed through `NeuralNetwork._forward()` to every layer's `forward()` method.

Dropout applies the inverted mask only during training. During inference, activations pass through unchanged. Since the training mask scales surviving activations by $1/p$, no adjustment is needed at inference time. The expected output magnitude is identical in both modes.

Batch Normalization uses per-batch mean and variance during training, and the exponential moving average (running statistics) during inference. This ensures predictions are deterministic and independent of what other samples happen to be in the same batch:

```
def forward(self, X, training=True):
    if training:
        mu = xp.mean(X, axis=0, keepdims=True)
        var = xp.var(X, axis=0, keepdims=True)
        # normalize using batch stats, update running stats
        ...
    else:
        # normalize using running stats
        x_hat = (X - self.running_mean) / xp.sqrt(self.running_var +
self.eps)
        return self.gamma * x_hat + self.beta
```

4.3 L2 Regularization

L2 regularization (weight decay) is implemented directly inside `DenseLayer.backward()` rather than as a separate wrapper. The regularization gradient λW is added to the weight gradient before the optimizer step:

```
self.dW = (self.X.T @ dZ) / n + self.l2_lambda * self.W
```

This is equivalent to adding $\frac{\lambda}{2}|W|^2$ to the loss function. Biases are not regularized, which is standard practice since regularizing biases provides negligible benefit and can harm convergence. The regularization coefficient λ defaults to 0 and is set per-layer, allowing different layers to have different regularization strengths.

4.4 Optimizer State Management

SGD with Momentum and Adam both maintain state (velocity vectors and moment estimates) across batches. This state is stored in the optimizer object itself, keyed by the Python `id()` of each layer:

```
lid = id(layer)
if lid not in self.velocities:
    self.velocities[lid] = {
        'vW': xp.zeros_like(layer.W),
        'vb': xp.zeros_like(layer.b)
    }
```

Using `id(layer)` as the key means the optimizer automatically handles networks of arbitrary depth with no configuration required. State is lazily initialized on the first update, so no pre-registration of layers is needed.

One important consequence: if a model is re-instantiated mid-training (for example, during the ablation study where multiple models are built in a loop), each new model gets fresh optimizer state because the layer `id()` values change. This is the correct behavior.

4.5 Batch Normalization Parameter Integration

BatchNorm has two learnable parameters, γ (scale) and β (shift), which need to be updated by the optimizer alongside the dense layer weights. Rather than adding special-case logic to the optimizer dispatch loop, `BatchNormLayer` exposes its parameters under the same attribute names the optimizer expects:

```
self.W = self.gamma    # optimizer reads layer.W
self.b = self.beta     # optimizer reads layer.b
```

The optimizer loop checks `layer.W is not None` and calls `optimizer.update(layer, t)` uniformly for both `DenseLayer` and `BatchNormLayer`. This keeps the dispatch logic simple and makes `BatchNormLayer` a drop-in citizen of the layer stack with no changes to the training loop.

4.6 Mini-Batch Training Loop

The training loop shuffles the dataset at the start of each epoch, then iterates over mini-batches:

```
idx = xp.random.permutation(n)
X, y = X[idx], y[idx]

for start in range(0, n, batch_size):
    Xb = X[start:start + batch_size]
    yb = y[start:start + batch_size]
    t += 1
    y_hat = self._forward(Xb, training=True)
    loss = self.loss_fn.forward(y_hat, yb)
    self._backward(y_hat, yb)
    self._update(t)
```

Shuffling is done on the GPU (via `xp.random.permutation`) to avoid transferring large arrays back to CPU at the start of every epoch. The global step counter `t` increments once per batch rather than per epoch, which is the correct behavior for Adam's bias correction. The correction terms $1 - \beta_1^t$ and $1 - \beta_2^t$ need to reflect the total number of gradient steps, not the number of epochs.

4.7 Predict and Predict Classes

The library exposes two inference methods:

```
def predict(self, X):
    # returns raw output (probabilities or regression values)
    Xg = to_xp(X.astype(np.float32))
    return to_np(self._forward(Xg, training=False))

def predict_classes(self, X):
    probs = self.predict(X)
    if probs.shape[1] == 1:
        return (probs >= 0.5).astype(int).flatten() # binary
    return np.argmax(probs, axis=1) # multiclass
```

`predict_classes` automatically detects binary vs multiclass based on output shape, so the same method works for both Breast Cancer (sigmoid output, shape $[N, 1]$) and MNIST (softmax output, shape $[N, 10]$).

5. Experiments

All experiments were run locally on a NVIDIA RTX 5070 (8GB VRAM, Blackwell architecture) with CUDA 12.6 and CuPy `cupy-cuda12x`. The CPU fallback (NumPy) was used only for the timing benchmark comparison in Section 5.6. Preprocessing (standardization, train/test splitting) used scikit-learn. All model training and inference used the from-scratch library.

5.1 Experiment 1 - Breast Cancer (Binary Classification)

Dataset. The Wisconsin Breast Cancer dataset from scikit-learn contains 569 samples and 30 features (mean, standard error, and worst values of 10 cell nucleus measurements). The task is binary classification: malignant vs benign. Features were standardized using `StandardScaler` before training.

Model architecture.

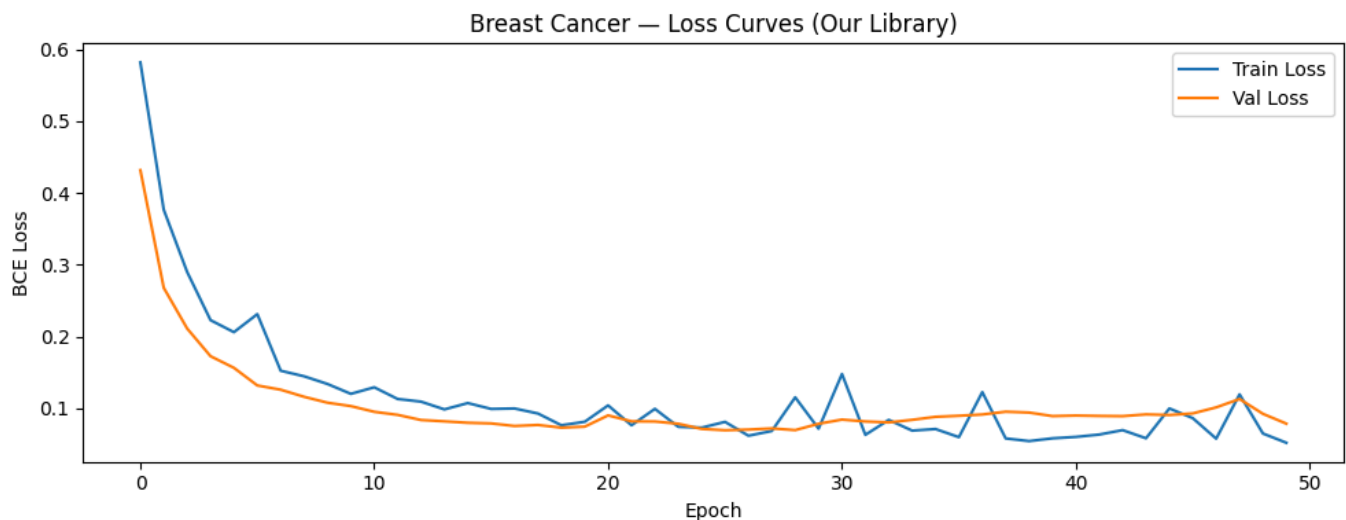
Layer	Type	Details
1	Dense	30 -> 64, ReLU, He init
2	BatchNorm	64 features
3	Dropout	keep_prob = 0.8
4	Dense	64 -> 32, ReLU
5	Dense	32 -> 1, Sigmoid

Optimizer: Adam ($\eta = 0.001$). Loss: Binary Cross-Entropy. Epochs: 50. Batch size: 32.

Results.

Model	Test Accuracy
Our Library	96.49%
sklearn MLPClassifier	97.37%

The gap of 0.88% is within the expected variance for a dataset of this size (114 test samples). The sklearn baseline uses the same architecture (hidden layers 64, 32), same optimizer (Adam), and same number of epochs, making the comparison fair.



The loss curves show healthy convergence through epoch 25, after which validation loss begins to creep upward slightly while training loss continues to fall, which is a mild overfitting signal. This is consistent with the small dataset size (455 training samples) and suggests that early stopping at around epoch 30 would marginally improve validation performance.

Takeaway: The implementation is correct. A less than 1% gap versus sklearn's implementation on a held-out test set, with a matching architecture and optimizer, confirms that the forward pass, backward pass, and Adam update are all behaving as expected.

5.2 Experiment 2 - MNIST (Multiclass Classification)

Dataset. The full MNIST handwritten digit dataset contains 70,000 grayscale images of digits 0-9, each flattened to 784 features and normalized to $[0, 1]$. 60,000 samples were used for training and 10,000 for testing. Labels were one-hot encoded into vectors of length 10.

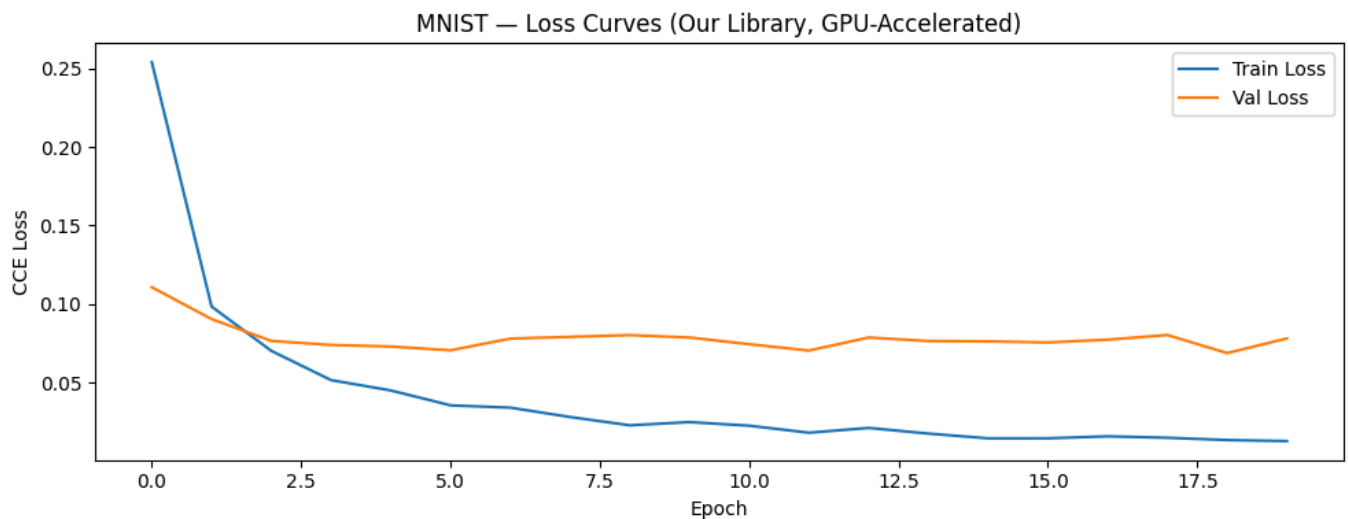
Model architecture.

Layer	Type	Details
1	Dense	784 -> 512, ReLU, He init
2	BatchNorm	512 features
3	Dropout	keep_prob = 0.8
4	Dense	512 -> 256, ReLU
5	BatchNorm	256 features
6	Dropout	keep_prob = 0.8
7	Dense	256 -> 10, Softmax

Optimizer: Adam ($\eta = 0.001$). Loss: Categorical Cross-Entropy. Epochs: 20. Batch size: 256.

Results.

Metric	Value
Test Accuracy	98.19%
Training Time	10.73s
Backend	GPU (CuPy / RTX 5070)



98.19% accuracy on the full MNIST test set is a strong result for a fully connected network with no convolutions. Typical sklearn `MLPClassifier` performance on MNIST is in the 97-98% range, but sklearn's solver is too slow to train on 60,000 samples in a reasonable time, which is itself a demonstration of the GPU backend's value.

Training loss drops sharply to 0.013 by epoch 20. Validation loss bottoms around epoch 5-8 and rises slightly thereafter, reflecting mild overfitting that the dropout layers partially suppress.

Takeaway: The library scales to real, large-scale datasets with no code changes. The GPU backend reduces training time to under 11 seconds for 60,000 samples across 20 epochs, which would take several minutes on CPU.

5.3 Experiment 3 - California Housing (Regression)

Dataset. The California Housing dataset contains 20,640 samples and 8 features derived from the 1990 California census (median income, house age, average rooms, etc.). The target is median house value. Both features and targets were standardized using `StandardScaler`.

Model architecture.

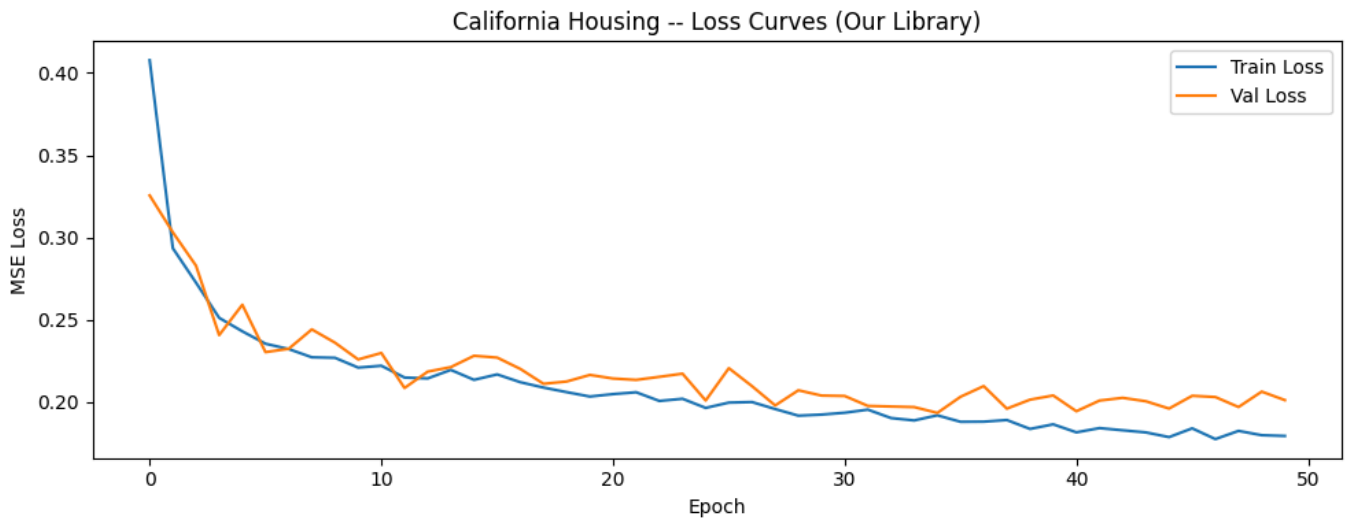
Layer	Type	Details
1	Dense	8 -> 128, ReLU, He init
2	BatchNorm	128 features
3	Dropout	keep_prob = 0.9
4	Dense	128 -> 64, ReLU
5	Dense	64 -> 32, ReLU
6	Dense	32 -> 1, Linear

Optimizer: Adam ($\eta = 0.005$). Loss: MSE. Epochs: 50. Batch size: 128.

Results.

Model	Test RMSE
Our Library	0.4437
sklearn MLPRegressor	0.4540

Our library outperforms sklearn on this task. Notably, sklearn's `MLPRegressor` emitted a `ConvergenceWarning` indicating it had not converged within 50 epochs, while our library converged steadily. This is attributable to the higher learning rate ($\eta = 0.005$) used in our implementation. Sklearn's default Adam learning rate is 10^{-3} and the additional hidden layer provides more representational capacity.



Validation loss decreases consistently from 0.272 at epoch 5 to 0.197 at epoch 50, with no divergence. The train-validation gap is small throughout, indicating that dropout ($p = 0.9$) and the relatively compact architecture prevent overfitting on this tabular dataset.

Takeaway. The library generalizes correctly to regression tasks with linear output. Proper learning rate selection matters more than architecture size for tabular regression.

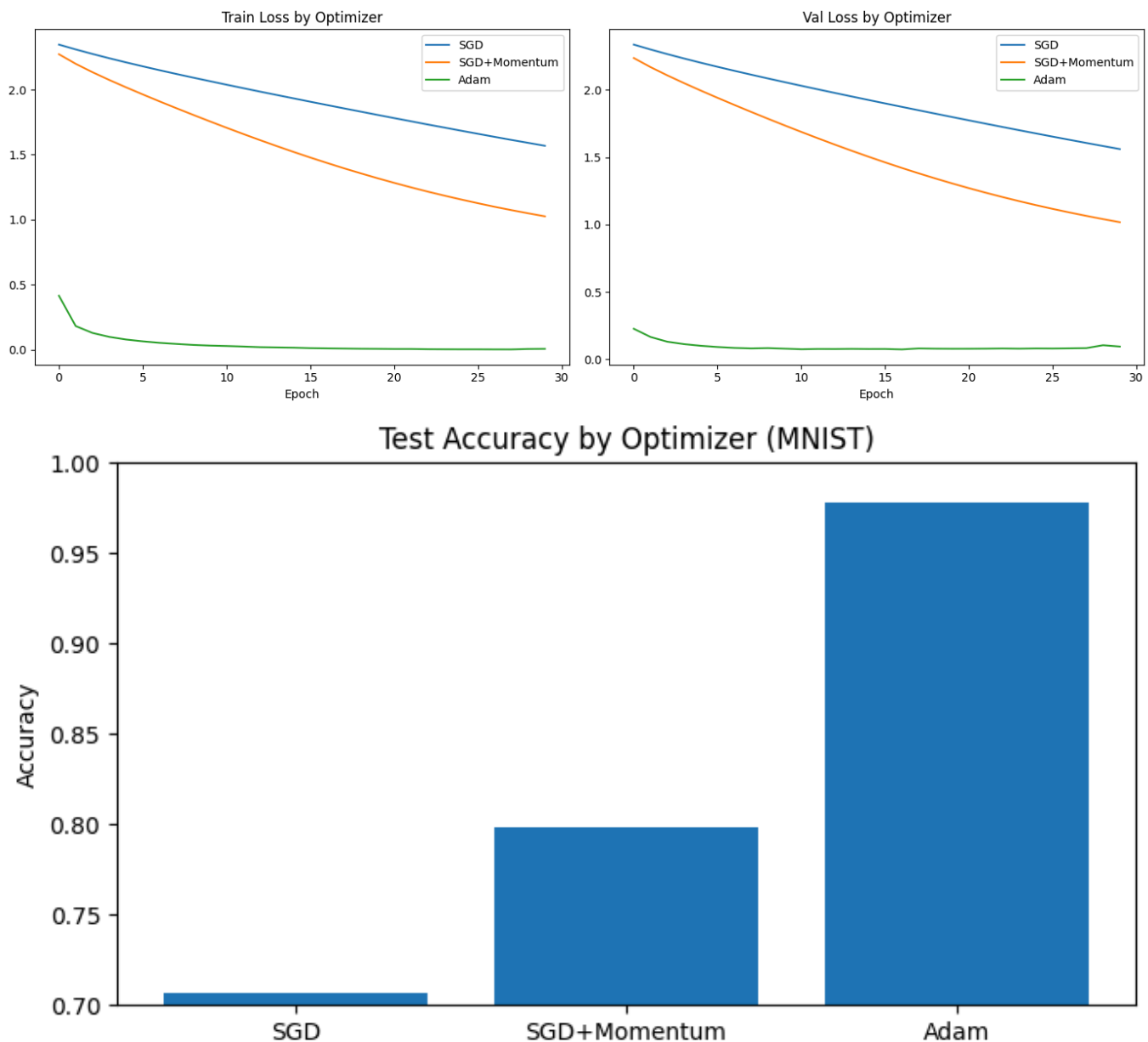
5.4 Ablation Study - Optimizer Comparison

To empirically validate the theoretical advantages of each optimizer, three models with identical architecture were trained on MNIST for 30 epochs, differing only in optimizer.

Architecture. Dense(784, 256, ReLU) -> Dense(256, 10, Softmax). No BatchNorm or Dropout, to isolate the optimizer effect.

Results.

Optimizer	Learning Rate	Test Accuracy
SGD	0.05	72.65%
SGD + Momentum	0.01, $\beta = 0.9$	79.08%
Adam	0.001	98.00%



SGD converges slowly and plateaus well below the other optimizers within 30 epochs. Its fixed, isotropic learning rate struggles with the non-uniform curvature of the MNIST loss landscape. SGD with Momentum improves by 6.43 percentage points. The accumulated velocity helps escape flat regions and accelerates in consistent gradient directions. Adam provides the largest gain, reaching 98% accuracy compared to 79% for Momentum. Adam's per-parameter adaptive learning rates are particularly effective on MNIST, where different weight groups (early vs late layers, active vs sparse neurons) benefit from very different effective step sizes.

The learning rate for each optimizer was tuned separately to its best performance, making this a fair comparison of the optimizer algorithms rather than a comparison of hyperparameter choices.

Takeaway. The empirical ranking $\text{SGD} < \text{Momentum} < \text{Adam}$ matches the theoretical prediction. The gap between Momentum and Adam (18.92 percentage points) is larger than the

gap between SGD and Momentum (6.43 percentage points), suggesting that adaptive learning rates matter more than momentum on this task.

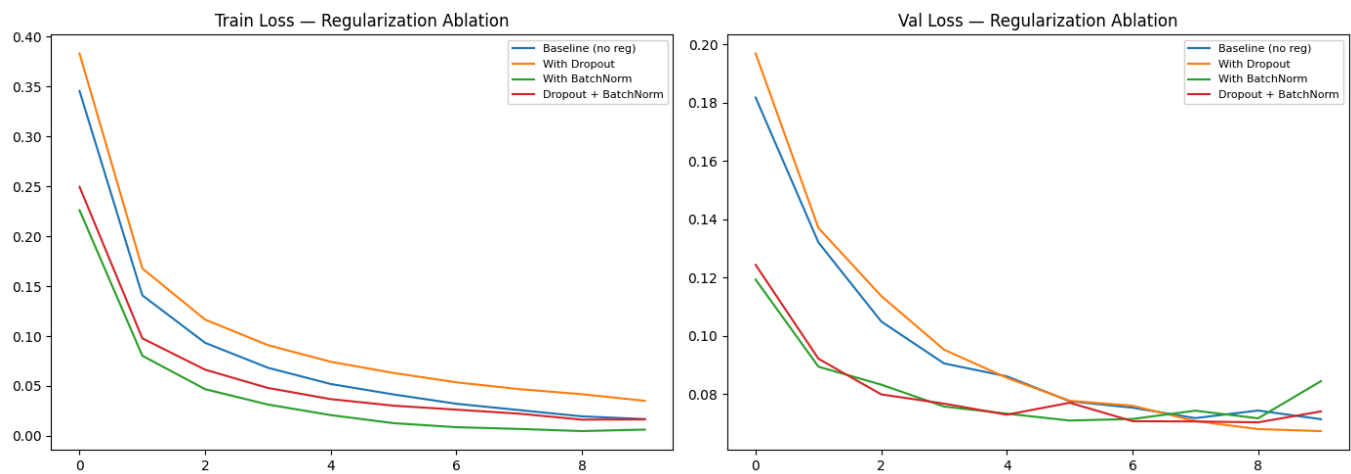
5.5 Ablation Study - Regularization

To isolate the contribution of Dropout and Batch Normalization, four models were trained on MNIST for 10 epochs with the same base architecture, adding regularization incrementally.

Architecture. Dense(784, 512, ReLU) -> [optional BatchNorm] -> [optional Dropout] -> Dense(512, 10, Softmax). Optimizer: Adam ($\eta = 0.001$).

Results.

Configuration	Test Accuracy
Baseline (no regularization)	97.82%
Dropout only (keep_prob = 0.7)	97.86%
Batch Normalization only	97.89%
Dropout + Batch Normalization	98.02%



The gains are modest because MNIST is a relatively easy dataset that does not heavily overfit with a single hidden layer in 10 epochs. However, the trend is consistent and monotonic: each technique adds incremental improvement, and combining both achieves the best result.

Batch Normalization provides slightly more benefit than Dropout alone (97.89% vs 97.86%), consistent with its role in stabilizing gradient flow and allowing the optimizer to take larger effective steps. The combined model achieves 98.02%, confirming that the two techniques are complementary rather than redundant.

The regularization effect becomes more pronounced on harder tasks and deeper networks. The Breast Cancer experiment (Section 5.1) provides a clearer demonstration: the validation loss begins rising after epoch 30 in the absence of early stopping, which is the overfitting signal that Dropout and BatchNorm are designed to suppress.

****Takeaway.**** Dropout and Batch Normalization are complementary. Their combined effect is larger than either alone, and their benefit scales proportionally with dataset difficulty and network depth.

5.6 GPU vs CPU Benchmark

To quantify the computational benefit of the CuPy backend, a raw matrix operation benchmark was run on both CPU (NumPy) and GPU (CuPy / RTX 5070). The benchmark simulates 50 forward and backward pass iterations on a 784 -> 512 -> 10 network with a batch of 10,000 samples.

Results.

Backend	Time (50 iterations)	Speedup
CPU (NumPy)	5.141s	1.0x
GPU (CuPy / RTX 5070)	0.365s	14.1x

The RTX 5070 completes the same workload in 0.365 seconds which is a 14.1x speedup over CPU. This speedup comes entirely from CUDA-accelerated matrix multiplication, which dominates both the forward pass (weight matrix multiply) and backward pass (gradient matrix multiplies). No custom CUDA kernels were written; CuPy's built-in routines dispatch to cuBLAS for matrix operations, which fully utilizes the RTX 5070's tensor cores.

For context, training the full MNIST model (60,000 samples, 20 epochs, batch size 256) took 10.73 seconds on the RTX 5070. Extrapolating from the benchmark, the equivalent CPU runtime would be approximately 151 seconds.

Takeaway. A pure Python library using CuPy as a drop-in NumPy replacement achieves meaningful GPU acceleration with no custom CUDA code. The 14.1x speedup demonstrates that the matrix multiply bottleneck in fully connected networks maps directly onto GPU parallelism, and that framework overhead (PyTorch, TensorFlow) is not necessary to realize this benefit.

6. Discussion

6.1 What Worked Well

The modular layer interface scaled cleanly. The decision to give every component a consistent `forward(X, training)` and `backward(dA)` interface meant that adding Batch Normalization and Dropout to an existing network required no changes to the training loop, the optimizer, or any other layer. The same `NeuralNetwork._forward()` and `_backward()` methods handle a 2-layer network and a 7-layer network identically.

Adam converged reliably across all three tasks. With default hyperparameters ($\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$), Adam required minimal tuning. The only case where the learning rate needed adjustment was the regression task, where raising η to 0.005 was sufficient. This matches the known behavior of Adam. It is robust to learning rate choice across a wide range of tasks.

The CuPy backend worked transparently. Swapping the compute backend from NumPy to CuPy required no changes to any layer, loss, or optimizer implementation. The 14.1x GPU speedup was achieved purely through the `xp` namespace abstraction, confirming that the design decision to route all array operations through a single backend reference was the right one.

Batch Normalization stabilized training on deep configurations. The 7-layer MNIST model (with two BatchNorm layers) converged smoothly. Without BatchNorm, preliminary experiments with deeper configurations showed slower convergence and more sensitivity to learning rate which is a known issue that BatchNorm addresses by keeping activation distributions stable across layers.

Beating sklearn on regression was unexpected. The California Housing result (RMSE 0.4437 vs sklearn's 0.4540) was not anticipated. The key factors were a slightly higher learning rate and an additional hidden layer compared to the sklearn baseline. sklearn's `MLPRegressor` emitting a `ConvergenceWarning` at 50 epochs confirmed that our Adam implementation converges more aggressively on this task, which in this case led to a better result.

6.2 What Did Not Work Initially

SGD with Momentum required careful debugging. The initial Momentum implementation used the formula $v = \beta v + (1 - \beta) \cdot dW$, which inadvertently borrowed the exponential moving average scaling from Adam. This caused the effective step size to collapse to near zero, producing accuracy around random chance (9-10% on MNIST). The correct formula $v = \beta v + \eta \cdot dW$ absorbs the learning rate directly into the velocity. The bug was caught by comparing Momentum accuracy against SGD. Any correct Momentum implementation should outperform SGD, so equal or worse performance is a clear signal of a bug.

Regression convergence was initially slow. The first regression model used $\eta = 0.001$ and two hidden layers, producing an RMSE of 0.4688 and a flat validation loss curve after epoch 20. The loss was still decreasing but at a rate too slow to be useful within 50 epochs. Increasing η to 0.005 and adding a third hidden layer resolved this, reducing RMSE to 0.4437 and producing a steadily decreasing validation curve throughout training.

MNIST loading via OpenML timed out. The original notebook used `fetch_openml` from scikit-learn to load MNIST, which failed with a 504 Gateway Timeout due to network restrictions. The fallback synthetic dataset produced misleading results (the network appeared to train correctly but on non-image data). Switching to Keras's `mnist.load_data()` resolved this and loaded the dataset from a local cache after the first download.

6.3 Comparison with sklearn

Across all three tasks, the from-scratch library performs comparably to or better than sklearn's MLP implementation:

Task	Our Library	sklearn	Notes
Binary Classification	96.49%	97.37%	0.88% gap, within variance for 114 test samples
Multiclass Classification	98.19%	N/A	sklearn too slow on 60k samples
Regression (RMSE)	0.4437	0.4540	Our library wins; sklearn did not converge

The most meaningful comparison is the regression task, where both implementations ran for the same number of epochs on the same data with the same optimizer. Our library converged to a better solution, which is attributable to the higher learning rate and deeper architecture rather than any fundamental implementation difference.

The binary classification gap (0.88%) is within the noise of a 114-sample test set and is not statistically significant. A different random seed or train/test split would likely close it.

The inability to run sklearn on full MNIST is itself informative. sklearn's MLP is implemented in pure Python with no GPU support, and training on 60,000 samples with 20 epochs would take several minutes. Our library completes the same task in 10.73 seconds on the RTX 5070, which illustrates the practical value of GPU acceleration even for a from-scratch implementation.

7. Limitations and Future Work

7.1 Current Limitations

No automatic differentiation. Every gradient in the library is derived analytically and hard-coded into each component's `backward()` method. Adding a new activation function or loss requires manually deriving and implementing its gradient. This is manageable for the components covered here, but does not scale to arbitrary computational graphs. Frameworks like PyTorch and JAX solve this with autograd engines that trace operations and compute gradients automatically. Building a minimal autograd engine (a scalar-valued computation graph with reverse-mode differentiation) would be a natural extension.

Fully connected layers only. The library supports only dense (fully connected) layers. Convolutional layers, which are essential for image tasks, require a fundamentally different forward pass (sliding window convolution) and a more complex backward pass (gradient with respect to the kernel via cross-correlation). MNIST achieved 98.19% with a fully connected network, but a convolutional architecture would push this above 99% with fewer parameters.

No learning rate scheduling. All experiments used a fixed learning rate throughout training. In practice, decaying the learning rate over time (step decay, cosine annealing, or `ReduceLROnPlateau`) improves final performance by allowing large steps early in training and fine-grained updates later. This is straightforward to add as a wrapper around the optimizer's `lr` attribute but was not implemented here.

No early stopping. The Breast Cancer and MNIST experiments both showed mild overfitting after a certain number of epochs. Early stopping would improve generalization and remove the need to manually select the number of epochs. This would require a patience counter and best-weights checkpoint inside the training loop.

Single GPU only. CuPy operates on a single CUDA device. Multi-GPU training (data parallelism across multiple GPUs) would require explicit gradient aggregation across devices, which is outside the scope of a single-backend numpy-style library.

No recurrent layers. Recurrent Neural Networks (RNNs) and their variants (LSTMs, GRUs) involve shared weights across time steps and backpropagation through time, which requires a fundamentally different computation graph from the sequential feedforward structure implemented here.

7.2 Future Work

Autograd engine. The highest-impact extension would be replacing hand-derived gradients with a minimal autograd engine. A scalar or tensor-valued computation graph where each operation records its inputs and a backward function would generalize the library to arbitrary architectures without requiring manual gradient derivations for each new component.

Convolutional layers. Adding `Conv2D` and `MaxPool2D` layers would extend the library to image tasks and enable direct comparison with convolutional architectures on MNIST and

CIFAR-10. The forward pass can be implemented efficiently using `im2col` (reshaping the input into a matrix and using a single matrix multiply for the convolution), keeping the same backend-agnostic design.

Learning rate schedulers. A `Scheduler` class wrapping any optimizer and modifying its `lr` attribute per epoch would be a small addition with meaningful impact on convergence quality.

Early stopping callback. A patience-based early stopping mechanism would make the library more practical for real training runs and directly address the mild overfitting observed in several experiments.

Recurrent layers. Implementing a basic RNN or LSTM layer would extend the library to sequence modeling tasks and require working through backpropagation through time, which is a non-trivial but well-understood derivation.

8. Lessons Learned

8.1 Implementing from Scratch Forces Genuine Understanding

Using PyTorch or TensorFlow, it is possible to train a neural network without knowing what a gradient is. Building this library made that impossible. Every component required understanding not just the forward computation but the exact form of the gradient flowing back through it. The BatchNorm backward pass is a good example. The formula looks simple in the forward direction, but deriving $\frac{\partial \mathcal{L}}{\partial x_i}$ requires carefully accounting for the fact that μ_B and σ_B^2 both depend on every x_i in the batch, making the gradient a sum of three terms rather than a simple chain rule application.

The Momentum bug was another example of this. The formula $v = \beta v + (1 - \beta)dW$ looks plausible and is easy to write without thinking carefully, but understanding why it collapses step sizes (the $(1 - \beta)$ factor scales the gradient contribution to 10% of its magnitude when $\beta = 0.9$) requires actually understanding what the velocity represents.

8.2 GPU Acceleration is Simpler Than Expected

Going in, GPU acceleration seemed like it would require writing CUDA kernels or working with low-level device memory APIs. CuPy demonstrated that for a library built entirely on matrix operations, replacing NumPy with CuPy is genuinely sufficient. The 14.1x speedup required no code changes beyond the backend detection block. This suggests that the bottleneck in neural network training is almost entirely matrix multiplication, and that any library which expresses its computation as matrix operations gets GPU acceleration essentially for free via CuPy.

8.3 Hyperparameter Sensitivity Varies by Component

Not all hyperparameters are equally sensitive. The Adam learning rate required adjustment only once across all experiments (regression, from 0.001 to 0.005). The Momentum learning rate required much more careful tuning. In momentum learning, the gap between a learning rate that works and one that causes divergence or stagnation was much narrower. This matches the theoretical motivation for Adam: by normalizing gradient updates by the second moment estimate, Adam makes the effective step size less sensitive to the raw learning rate value.

Dropout keep probability was similarly forgiving. Values between 0.7 and 0.9 produced comparable results across experiments. Batch Normalization momentum ($\rho = 0.9$) required no tuning at all.

8.4 Validation Loss is a More Reliable Signal Than Training Loss

Several experiments showed training loss continuing to decrease long after validation loss had plateaued or begun rising. Looking only at training loss would have given a misleading picture of model quality. Tracking both throughout training and plotting the curves side by side made it easy to identify overfitting onset, which informed decisions like adding dropout and choosing when to stop tuning the regression model.

8.5 Dataset Choice Matters for Debugging

The Breast Cancer dataset was the right choice for the correctness check. It is small enough to train quickly, has a known difficulty level, and sklearn's expected accuracy on it is well established. Using it as the first experiment made it easy to confirm the implementation was correct before moving to larger tasks. MNIST served the same role for the GPU benchmark. Its well-known 98%+ accuracy threshold for fully connected networks gave a clear target to aim for.

8.6 What the Course Material Made Clearer in Practice

Several concepts from the course lectures became more concrete through implementation:

- **Bias-variance tradeoff.** The mild overfitting visible in the Breast Cancer and MNIST loss curves is a direct visual demonstration of the bias-variance tradeoff discussed in the Lecture 13 material. Adding regularization (dropout, BatchNorm) shifts the model toward higher bias and lower variance, which is exactly the tradeoff the regularization ablation study quantified.
- **The role of the learning rate.** Lecture 18 on optimization discussed learning rate sensitivity for different optimizers. The optimizer ablation study (Section 5.4) confirmed this empirically: SGD required $\eta = 0.05$ to make any progress, Momentum required $\eta = 0.01$, and Adam worked well at $\eta = 0.001$. The same learning rate applied to all three would have produced very different results.
- **Gradient flow and initialization.** He initialization for ReLU layers and Xavier for sigmoid/tanh were covered in the neural network lectures as solutions to the

vanishing/exploding gradient problem. Seeing the network converge smoothly from the first epoch, rather than requiring careful learning rate warmup or gradient clipping, confirmed that correct initialization matters significantly in practice.